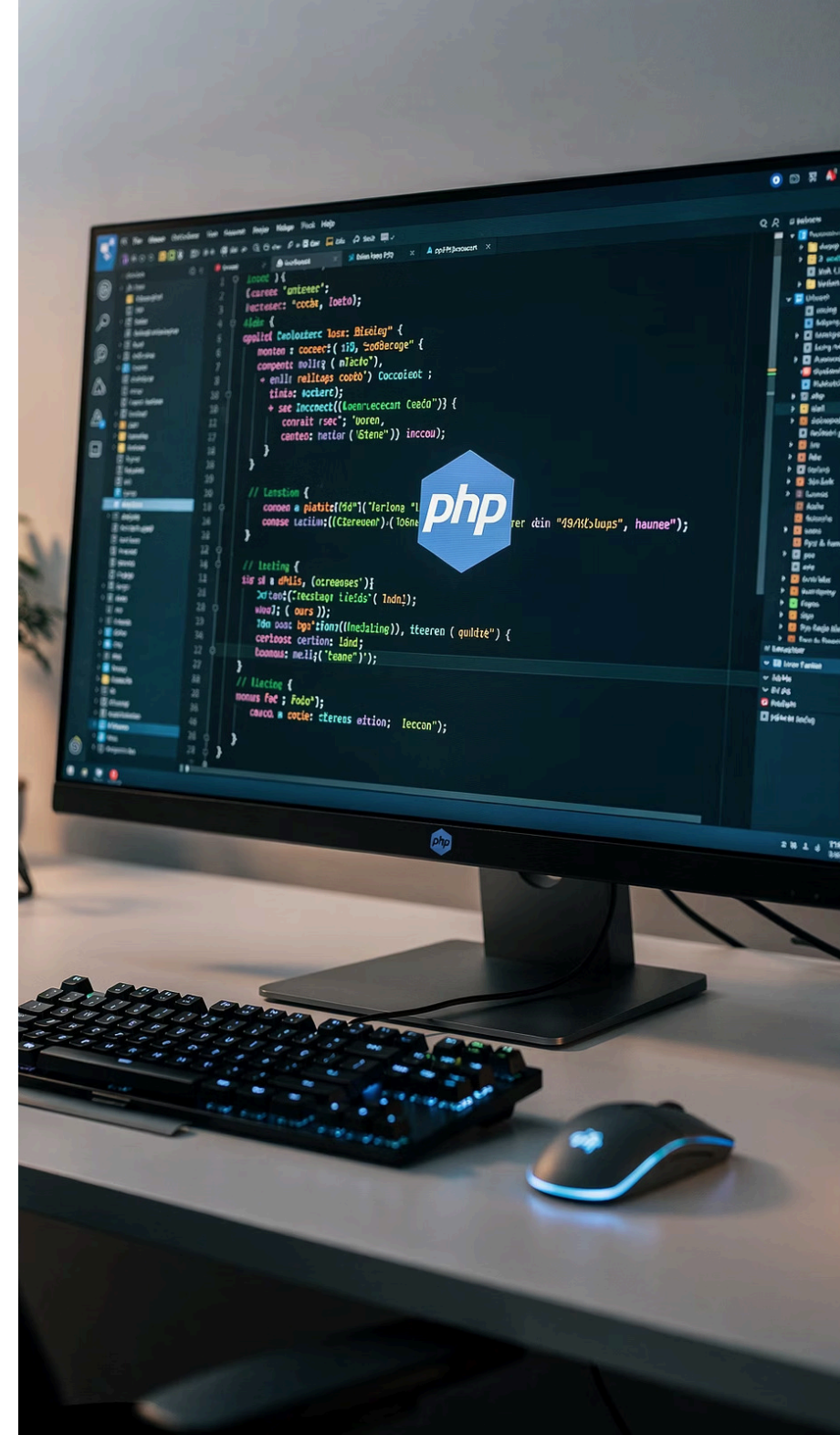


Topic 1: Development Environment & Introduction to PHP

Welcome to the foundational module of your PHP journey. Before you can build dynamic, data-driven websites, you need the right tools in place. This topic walks you through everything from setting up your local environment to writing and running your very first PHP program.

LEARNING OUTCOME

By the end of this topic, you will be able to **apply the basics of PHP to develop dynamic websites** understanding the server-side flow, creating project files, and producing output in a browser using PHP syntax.



Part 1: Tools Required for PHP Development

Before writing a single line of PHP, you need to set up a **local development environment**. This is the collection of software tools that lets your computer act like a web server - so you can build and test PHP websites without needing to go online. There are three essential categories of tools every PHP developer needs.

1.1 Code Editor / IDE

A code editor is where you write, read, and edit your source code. A good editor makes coding faster and reduces mistakes.

- **Visual Studio Code (VS Code)** 
Recommended
- Sublime Text
- Notepad++

Why VS Code?

VS Code has become the industry standard for beginner and professional developers alike. Here's why it stands out:

- **Free and lightweight** - runs smoothly on most computers
- **Multi-language support** - PHP, HTML, CSS, and JavaScript all in one place
- **Extensions** - install **PHP Intelephense** for smart code suggestions, error highlighting, and auto-completion
- **Integrated terminal** - run commands without switching windows

Download VS Code for free at **code.visualstudio.com**.

1.2 Web Browser

A web browser is the tool you use to **view and test your PHP web application**. Once your PHP code has been processed by the server, the resulting HTML is sent to the browser for display - just like any regular website. You interact with your project through the browser as if it were a live website.



Google Chrome

The most widely used browser worldwide. Chrome's **DevTools** (press F12) allow you to inspect HTML elements, view console errors, monitor network requests, and debug your PHP-generated output in real time. Highly recommended for development.



Mozilla Firefox

An excellent open-source browser with powerful built-in developer tools. Firefox is privacy-focused and a popular alternative to Chrome. Its **Responsive Design Mode** is great for testing mobile layouts of your PHP web pages.

  **Pro Tip:** Always test your PHP projects in at least one modern browser. Both Chrome and Firefox are free to download and offer excellent developer tools to help you catch and fix issues quickly.

1.3 Local Server Environment

This is the most critical tool for PHP development. Unlike HTML files - which you can open directly in a browser - **PHP must be processed by a web server** before it can be displayed. PHP is a **server-side language**, meaning the code runs on the server, not in the browser.

A local server environment installs a mini web server on your own computer, letting you simulate a real hosting environment without needing an internet connection or paid hosting account.

XAMPP Recommended

A free, cross-platform package (Windows, Mac, Linux) that bundles Apache, MySQL, PHP, and phpMyAdmin together. Ideal for beginners due to its simple Control Panel interface. We will use XAMPP throughout this course.

WAMP

Short for **Windows + Apache + MySQL + PHP**. A Windows-only solution. Similar to XAMPP but exclusive to Windows users. A solid alternative if XAMPP causes compatibility issues on your machine.

MAMP

Short for **Mac + Apache + MySQL + PHP**. Originally designed for macOS, though a Windows version exists. MAMP is popular among Mac users and has a clean, user-friendly interface.

What XAMPP Contains

XAMPP is not just one program - it's a **bundle of four essential components** that work together to create a complete local web server environment. Each component has a specific role, and together they provide everything a beginner needs to run PHP applications locally.



Apache

The **web server** that handles HTTP requests. When you type `localhost` in your browser, Apache responds and serves your PHP files.



MySQL

The **database management system** used to store, retrieve, and manage data for your web applications. Essential for dynamic, data-driven sites.



PHP

The **PHP interpreter** that reads your `.php` files and executes the server-side code, converting it into HTML output for the browser.



phpMyAdmin

A **graphical user interface** for managing your MySQL databases. Instead of typing SQL commands, you can use a visual dashboard - perfect for beginners.

- ❏ XAMPP bundles all four tools together with a single installer, making it **the easiest way for beginners to get started** with PHP development without complex configuration.

Part 2: Installation & Setup

2.1 Installing XAMPP

Installing XAMPP is straightforward and takes only a few minutes. Follow the steps below carefully to ensure everything is configured correctly before you start coding.

Installation Steps

1. Visit the official XAMPP website:
apachefriends.org
2. Download the version for your operating system (Windows, macOS, or Linux)
3. Run the installer and follow the on-screen instructions. Accept the default settings unless you have a specific reason to change them.
4. Once installed, open the **XAMPP Control Panel** from your Start menu or Applications folder

Starting Your Services

In the XAMPP Control Panel, you will see a list of modules. You need to start two key services:

- Click **Start** next to **Apache** this activates the web server
- Click **Start** next to **MySQL** this activates the database server

✓ When both services turn **green**, your local development environment is live and ready. You can now open your browser and navigate to `http://localhost` to confirm Apache is running correctly.



⚠ If a service fails to start, another application may be using the same port. Common conflict: Skype can block Port 80 used by Apache.

2.2 Understanding the Project Folder

Once XAMPP is installed, it's important to understand **where your PHP project files must be saved**. PHP files cannot be run from just any folder on your computer - they must be placed inside a specific directory that Apache monitors and serves.

1

Locate the htdocs Folder

After installing XAMPP, navigate to the following path on your computer:

`C:\xampp\htdocs\`

This is the **root directory** of your local web server. Apache serves all files located inside this folder.

2

Why htdocs?

The `htdocs` folder stands for **HyperText Documents**. It is the official document root for the Apache server in XAMPP. Any file or folder you create inside `htdocs` becomes accessible through the browser using `http://localhost/`.

3

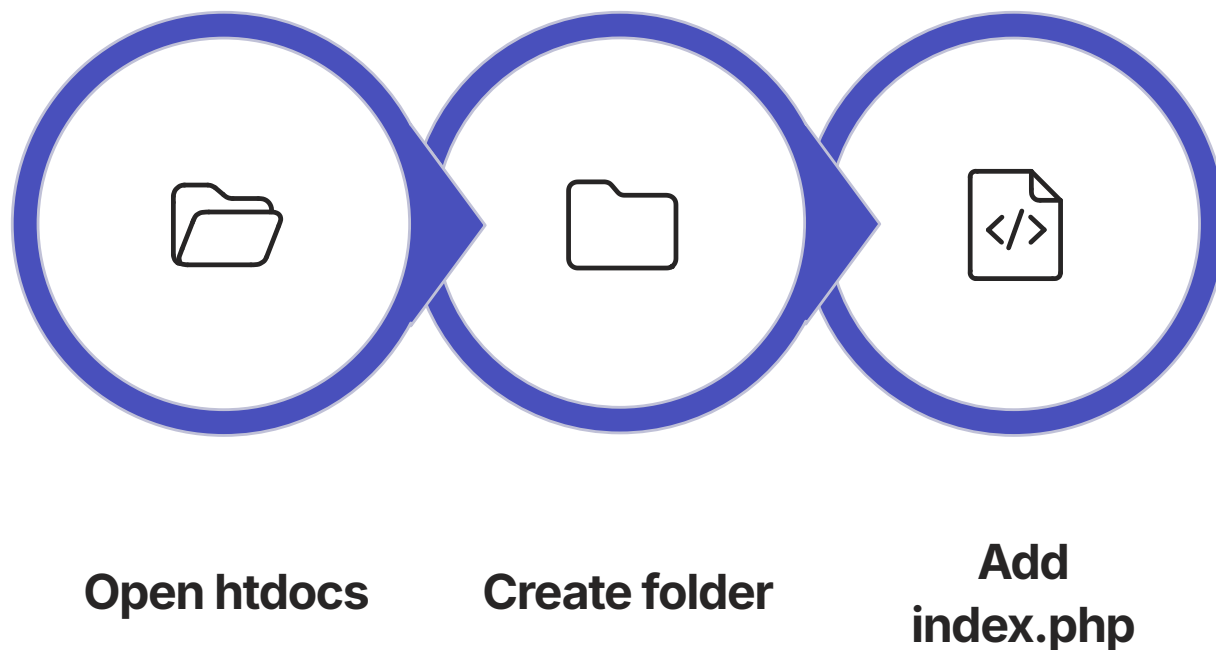
All Projects Go Here

Every PHP project you create during this course and in your career must be stored inside a dedicated subfolder within `htdocs`. For example:

`C:\xampp\htdocs\myproject\` becomes accessible at `http://localhost/myproject/`.

2.3 Creating Your First Project

Now that XAMPP is running and you understand the `htdocs` folder, it's time to create your very first PHP project. This involves just two simple steps: creating a project folder, and creating a PHP file inside it.



Following this structure keeps your projects organized and ensures Apache can find and serve your files correctly. The `index.php` file is the **default entry point** - when you visit `http://localhost/myproject/`, Apache automatically loads `index.php` first, just like `index.html` on a traditional website.

Folder Naming Tips

- Use lowercase letters only (e.g., `myproject`)
- Avoid spaces - use hyphens or underscores instead (e.g., `my-project`)
- Keep names short and descriptive

File Naming Tips

- PHP files must end with the `.php` extension
- The main page is always named `index.php`
- Additional pages can be named descriptively (e.g., `about.php`, `contact.php`)

2.4 Running Your PHP Project

With your project folder and `index.php` file created inside `htdocs`, you're ready to run it. This step is where PHP development feels different from HTML - you don't just open the file directly; you **request it through the browser via the local server**.

1 Make Sure Apache is Running

Open the XAMPP Control Panel and confirm that Apache shows a green status. If it's not running, click **Start** next to Apache. Without Apache, your PHP code cannot be processed.

2 Open Your Browser

Launch Google Chrome or Mozilla Firefox. Do **not** use File > Open to navigate to your PHP file - this bypasses the server entirely and PHP code will not execute.

3 Type the Localhost URL

In the address bar, type: `http://localhost/myproject/` then press Enter. Apache receives this request, locates your `index.php` file, processes the PHP code, and sends the resulting HTML back to your browser for display.

 **Key Concept:** The URL `http://localhost/` always points to `C:\xampp\htdocs\`. So `http://localhost/myproject/` maps directly to `C:\xampp\htdocs\myproject\`. This mental model is essential for understanding how PHP web servers work.

Part 3: First PHP Program

3.1 Basic Example Hello, World!

Every programming language is traditionally introduced with a "Hello, World!" program. This simple example demonstrates the essential structure of a PHP script and confirms that your environment is working correctly. If you see the output in the browser, your setup is complete.

```
<?php  
echo "Hello, World!";  
?>
```

Breaking It Down

1. `<?php` - Opens the PHP block. Everything inside is processed by the server.
2. `echo` - The PHP command used to output text to the browser.
3. `"Hello, World!"` - The string value to display. Strings must be wrapped in quotes.
4. `;` - Every PHP statement must end with a semicolon.
5. `?>` - Closes the PHP block. Apache stops processing PHP here.

When you run this file in the browser, the output will simply show: **Hello, World!** — with no PHP tags visible, because PHP is server-side.

3.2 Mixing PHP with HTML

One of the most powerful features of PHP is its ability to be **embedded directly inside HTML**. This is exactly how real-world web applications are built - a single `.php` file can contain both HTML structure and dynamic PHP logic, seamlessly combined.

```
<html>
  <body>
    <h1><?php echo "Welcome to PHP Class"; ?></h1>
  </body>
</html>
```

How It Works

In this example, the file contains standard HTML tags (`<html>`, `<body>`, `<h1>`) with a PHP block embedded inside the `<h1>` tag. When Apache processes the file:

1. All plain HTML is passed through unchanged
2. The PHP block `<?php echo "Welcome to PHP Class"; ?>` is executed
3. The output of `echo` replaces the PHP block
4. The final HTML sent to the browser looks like a normal HTML page

Why This Matters

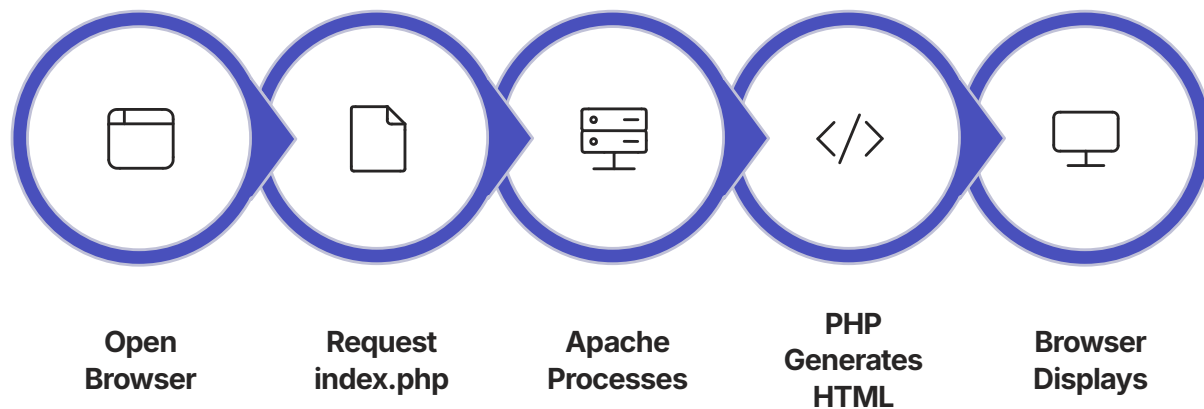
This mixing capability is what makes PHP so practical for web development. Instead of writing separate files for structure and logic, you can build a **single dynamic template** that outputs different content depending on user input, database results, or application state.

For example, a product page could display different prices, names, and images based on which product the user clicked - all from one `.php` template file pulling data from a database.

 You can open and close PHP tags (`<?php ... ?>`) as many times as needed within a single HTML file.

Part 4: How PHP Works

Understanding the flow of a PHP request is one of the most important conceptual foundations in web development. This process known as the **request response cycle** explains why PHP must run on a server, and how the browser ultimately receives and displays content.



This flow happens almost instantly every time a user visits a PHP-powered web page. The key takeaway: **PHP is invisible to the end user**. By the time content reaches the browser, it has already been transformed into standard HTML. The user never sees PHP code only its output.



Key Insight: Server vs. Browser

One of the most fundamental concepts in web development - and a common source of confusion for beginners - is the distinction between **server-side** and **client-side** code. PHP and HTML play very different roles in this system.

PHP Runs on the Server

PHP code is executed entirely on the **web server** (Apache, in our case). The server reads the `.php` file, runs the PHP instructions, and produces an output - typically HTML. The PHP source code is **never transmitted** to the browser. This makes PHP ideal for handling sensitive operations like database queries, password validation, and business logic.

- Processes on the server (Apache + PHP interpreter)
- Has access to files, databases, and server resources
- Hidden from the end user

HTML Runs in the Browser

HTML (along with CSS and JavaScript) is executed and interpreted by the user's **web browser**. After the PHP server has finished processing and generated HTML, that HTML is sent across the network to the browser, which renders it as a visual web page. The browser never knows PHP was involved.

- Rendered on the client (user's browser)
- Visible to users via "View Page Source"
- Controls appearance and interactivity

Common Errors & Troubleshooting

When starting out with PHP, you will almost certainly encounter errors. This is completely normal and part of the learning process. Below are the three most common issues beginners face, along with their causes and solutions.

Error	Cause	Solution
Page Not Found (404)	Your PHP file is saved in the wrong folder - not inside <code>htdocs</code>	Move your project folder to <code>C:\xampp\htdocs\</code> and reload the browser
PHP Code Not Running	Apache is not started in the XAMPP Control Panel	Open XAMPP, click Start next to Apache, wait for the green status indicator
Blank / White Page	There is a syntax error in your PHP code (e.g., missing semicolon or quote)	Check your code carefully for typos, missing <code>;</code> or unmatched quotes. Enable error reporting in PHP settings.

 **Tip:** To display PHP errors in the browser during development, add this line at the top of your PHP file: `<?php error_reporting(E_ALL); ini_set('display_errors', 1); ?>` - this is invaluable for debugging as a beginner.

TOPIC 2 BEGINS

Topic 2: Control Structures & Arrays in PHP

Learning Outcome (2.1.2): Employ conditional statements, iterations, and arrays for data processing in dynamic websites.

Part 1: Introduction to Control Structures

Control structures are among the most powerful features of any programming language. They give your program the ability to **respond intelligently to different situations** - rather than blindly executing the same instructions every time, a program with control structures can make decisions, repeat tasks, and manage complex data.

In simple terms: **Control structures control the flow of execution** in a program. They determine *which* code runs, *when* it runs, and *how many times* it runs. Without control structures, every PHP script would do exactly the same thing every time - making dynamic websites impossible.

Make Decisions

Execute different blocks of code depending on whether a condition is true or false.
Example: Show a welcome message only if the user is logged in.

Repeat Actions

Execute a block of code multiple times without writing it repeatedly.
Example: Display all 100 products from a database with just a few lines of code.

Manage Data

Store, access, and process collections of related data efficiently using arrays.
Example: Store a shopping cart of items and iterate through them at checkout.

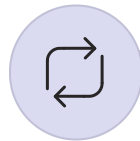
Types of Control Structures in PHP

PHP supports three major categories of control structures, each serving a distinct purpose in program logic. Together, they form the foundation of all decision-making and data handling in PHP applications.



Arrays Data Storage & Processing

Arrays are special variables that can hold **multiple values** under a single name. Instead of creating ten separate variables for ten items, you use one array. Arrays are essential for managing lists, tables of data, form inputs, and database results in PHP.



Iteration Statements (Loops)

Loops allow you to **repeat a block of code** multiple times. PHP supports `for`, `while`, `do...while`, and `foreach` loops. They are used to process arrays, generate HTML tables, and automate repetitive tasks.



Conditional Statements Decision Making

These structures evaluate a **condition** (a true/false test) and execute different code depending on the result. PHP supports `if`, `if...else`, `if...elseif...else`, and `switch` statements. This is where program logic begins.

Part 2: Conditional Statements

Conditional statements are the foundation of decision-making in PHP. They allow your program to **evaluate a condition** - a question with a true or false answer - and then execute different code depending on the result. This is how PHP creates truly dynamic, responsive web pages.

Think of a conditional statement like a traffic light: if the light is green, you go; if it's red, you stop. Your PHP program makes the same kind of binary decisions constantly - checking user roles, validating form data, comparing values, and branching into different paths of execution.

Why Conditionals Matter

Without conditional statements, every user visiting your website would see the same content and experience the same behavior - regardless of who they are, what they've submitted, or what's in the database.

Conditionals are what make personalization, validation, and access control possible.

Conditions in PHP

A **condition** is any expression that evaluates to either `true` or `false`. Common examples include:

- `$age >= 18` - is the user 18 or older?
- `$password == "secret"` - does the password match?
- `$score > 50` - did the student pass?

2.1 The IF Statement

The `if` statement is the most basic and most widely used conditional in PHP. It evaluates a condition and executes a block of code **only if that condition is true**. If the condition is false, the block is skipped entirely and execution continues after the closing brace.

Syntax

```
if (condition) {  
    // code executes if  
    // condition is TRUE  
}
```

Example

```
<?php  
$age = 18;  
if ($age >= 18) {  
    echo "You are an adult";  
}  
?>
```

Step-by-Step Explanation

1. **Variable defined:** `$age = 18` stores the value 18
2. **Condition evaluated:** Is `$age >= 18`? In this case, `18 >= 18` is **TRUE**
3. **Code executes:** Since the condition is TRUE, the `echo` statement runs and "You are an adult" is displayed
4. If `$age` were 16, the condition would be FALSE, and *nothing* would be displayed

Common Comparison Operators

- `==` - Equal to
- `!=` - Not equal to
- `>` - Greater than
- `<` - Less than
- `>=` - Greater than or equal to
- `<=` - Less than or equal to

2.2 The IF...ELSE Statement

The `if...else` statement extends the basic `if` statement by providing a **second block of code to execute when the condition is false**. This ensures your program always has an action - whether the condition is true or false - making it ideal for handling two possible outcomes.

Example

```
<?php
$marks = 40;
if ($marks >= 50) {
    echo "Pass";
} else {
    echo "Fail";
}
?>
```

Output

Since `$marks = 40` and `40 >= 50` is **FALSE**, the `else` block runs and the browser displays: **Fail**

How IF...ELSE Works

1. The condition `$marks >= 50` is evaluated
2. If **TRUE** the `if` block runs: displays "Pass"
3. If **FALSE** the `else` block runs: displays "Fail"
4. Only one block ever executes never both

Real World Uses

- Show "Welcome back!" or "Please log in" depending on login status
- Display "In Stock" or "Out of Stock" based on inventory count
- Show "Admin Panel" or "Access Denied" based on user role
- Output "Pass" or "Fail" based on a student's score



Tip: The `else` block is a safety net - it guarantees your program always has a defined response, even when the primary condition isn't met.

Topic Summary & Key Takeaways

You've now covered the essential foundations for PHP web development. Let's review the key concepts from both topics to consolidate your understanding before moving forward.

01

Set Up Your Environment

Install **VS Code** as your code editor, use **Google Chrome** for testing, and run **XAMPP** as your local server. Always start Apache and MySQL before developing.

02

Understand the htdocs Folder

All PHP projects must be stored in `C:\xampp\htdocs\`. Access them in the browser via `http://localhost/projectname/`. PHP files require a server - never open them directly.

03

Know the PHP Request Flow

The browser requests a `.php` file
→ Apache processes it with PHP
→ PHP outputs HTML → the browser displays the result. PHP is server-side and invisible to the user.

04

Write Basic PHP Syntax

Use `<?php` and `?>` tags. Use `echo` to display output. End every statement with a semicolon `;`. PHP can be mixed inside HTML documents.

05

Apply Conditional Statements

Use `if` for single-condition checks. Use `if...else` for two possible outcomes. Conditions evaluate to `true` or `false` and control which block of code executes.